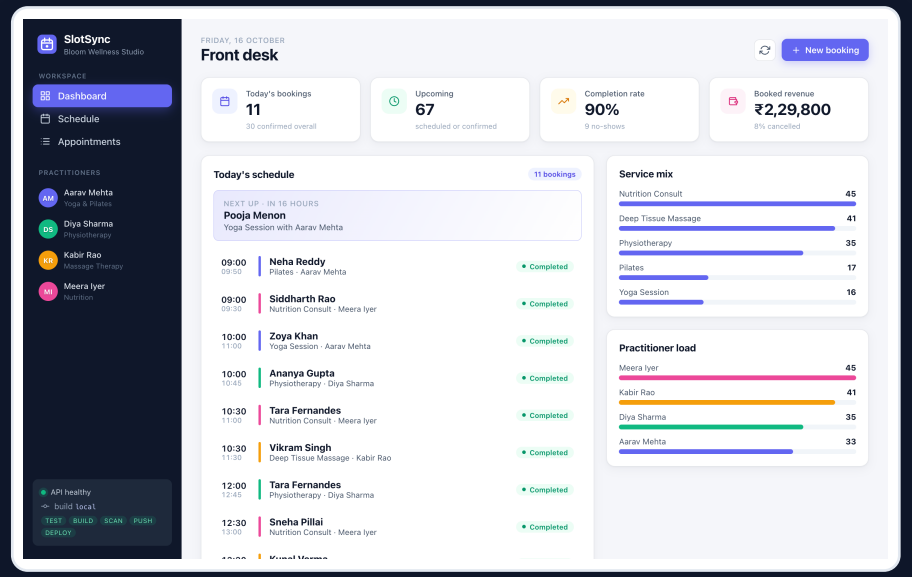
SlotSync

Appointment booking, from laptop to a monitored Kubernetes cluster



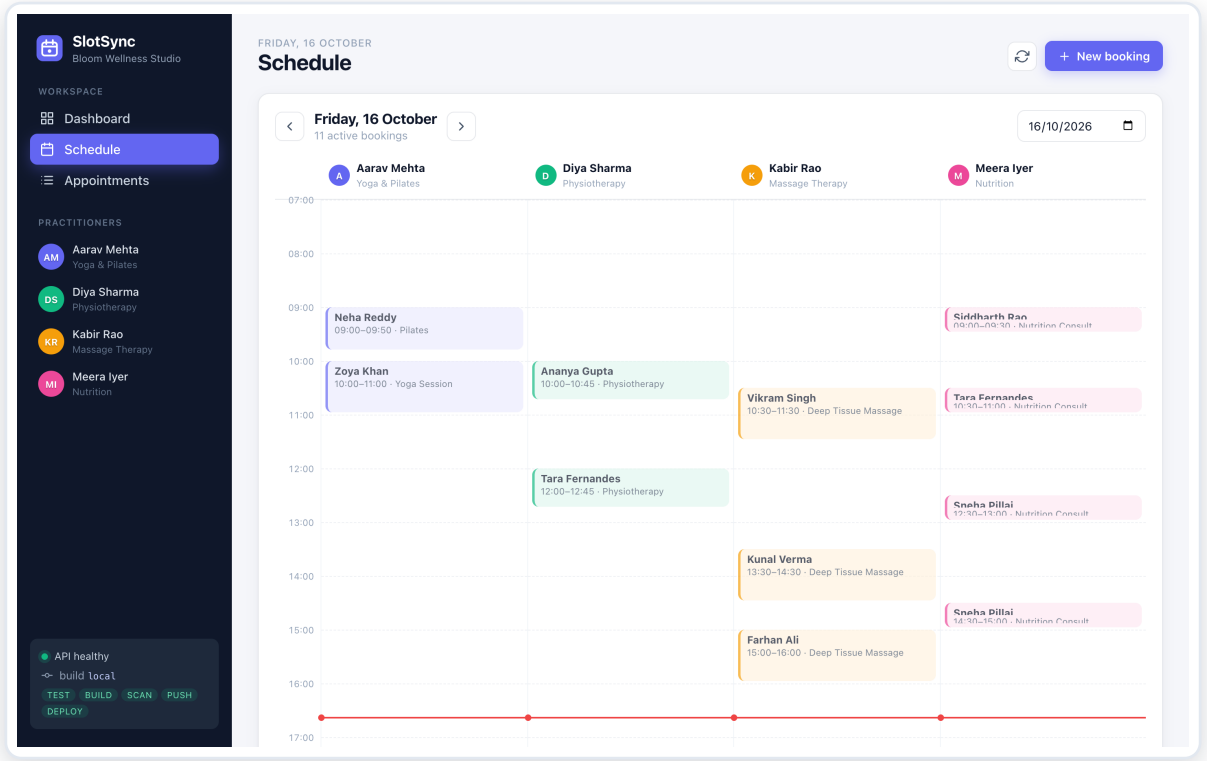
DevOps Final Capstone

Anushka Jain · Scaler School of Technology · October 2026

github.com/anushkabhansalii/apoointment-booking-devops-project

What I built and why

- **The problem:** a 4-practitioner wellness studio double-books rooms and has no view of the day
- **The app:** book, reschedule, confirm and cancel sessions; the API refuses double-bookings and out-of-hours slots
- **The platform:** every push is tested, scanned, packaged, published and rolled out to EKS by GitOps, then monitored
- **Ground rules:** nothing clicked in a console, no secrets in Git, every artefact traceable to a commit SHA



13
REST endpoints

32
automated tests

61
AWS resources in code

Tech stack

Application

React 18 + Vite · FastAPI · SQLAlchemy 2 · Pydantic v2

Data

PostgreSQL 16 · Alembic migrations · gp3 EBS volume

Quality

pytest + coverage gate · Ruff · ESLint · alembic check

Containers

Multi-stage Dockerfiles · non-root · Docker Compose

CI/CD

GitHub Actions · GHCR · multi-arch images · SHA tags

DevSecOps

Trivy · Gitleaks · SBOM + provenance · Pod Security

Infrastructure

Terraform · AWS VPC · NAT · EKS 1.35 · Pod Identity

Kubernetes

Helm chart · Ingress · HPA · PDB · NetworkPolicy

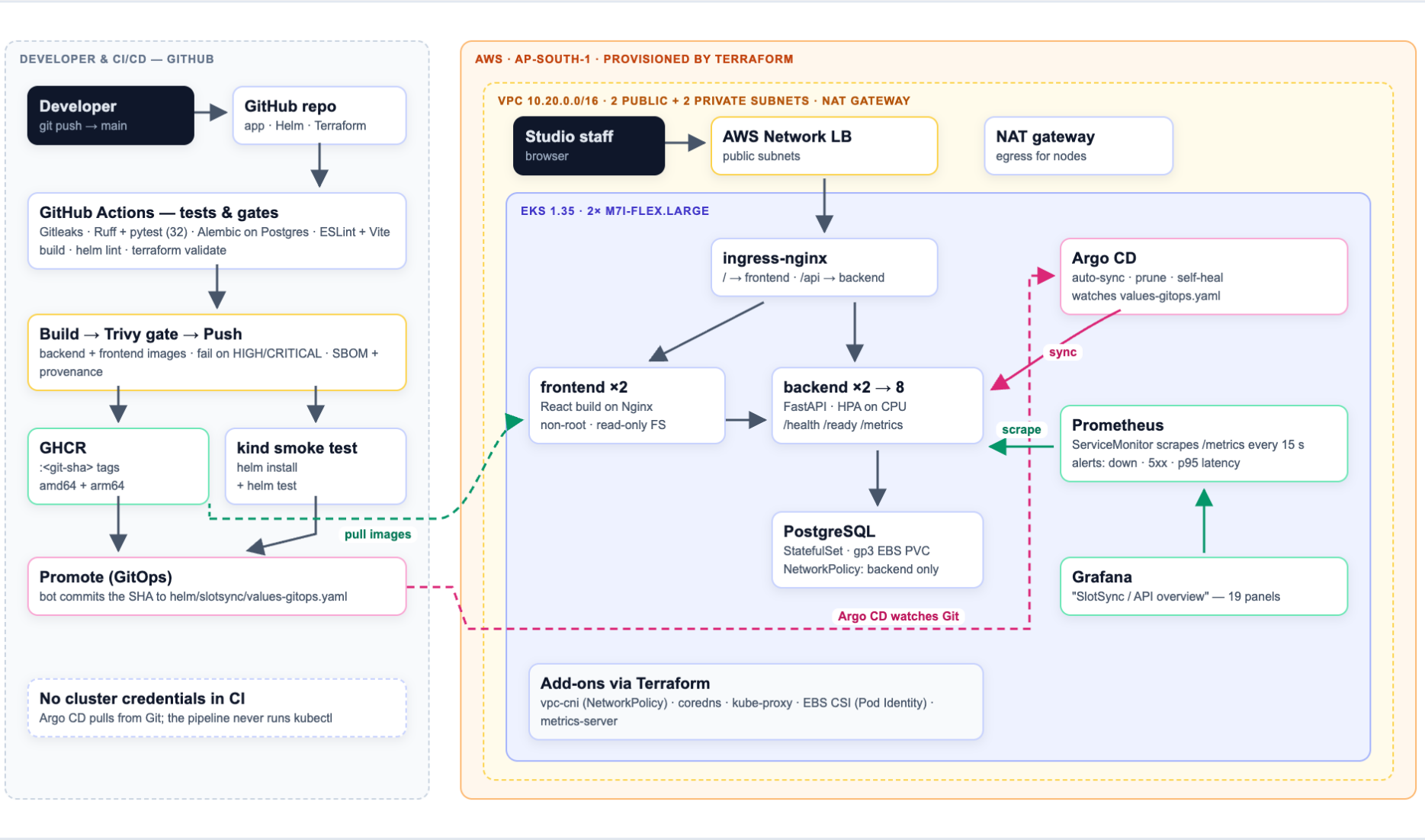
GitOps

Argo CD · automated sync, prune, self-heal

Observability

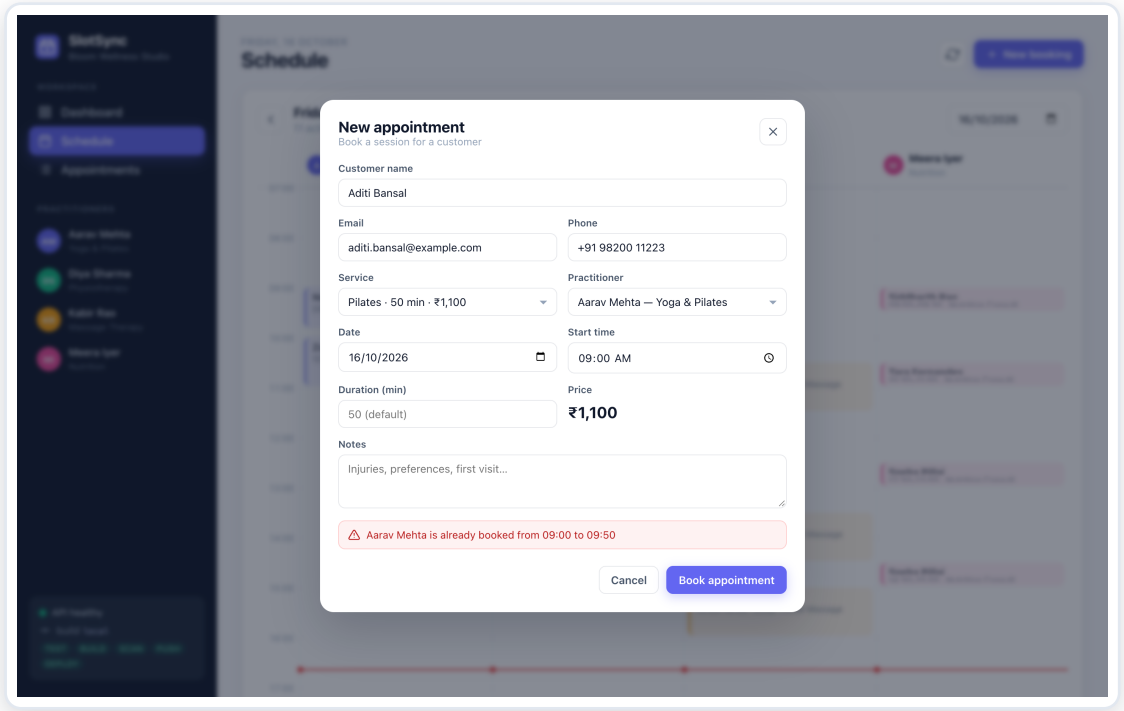
Prometheus · Grafana · ServiceMonitor · alert rules

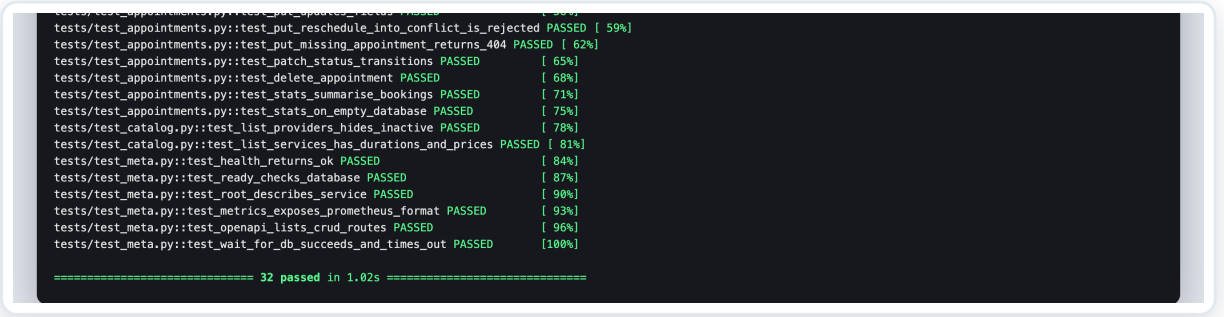
Architecture



Application and API

- **FastAPI:** CRUD on /api/appointments, status PATCH, stats, providers, services
- **Probes:** /health for liveness, /ready runs SELECT 1, /metrics for Prometheus
- **Rules:** 409 on double-booking, 422 outside 07:00–21:00, UTC storage, IST display
- **Database:** Alembic revision 0001 creates both tables and seeds practitioners; advisory lock stops replicas racing
- **Tests:** 32 pytest cases on an isolated in-memory DB, 97% coverage, warnings are errors





Containers

Backend image

- python:3.12-slim, multi-stage venv
- pip and setuptools removed at runtime
- UID 10001, waits for DB, migrates, runs Uvicorn

Frontend image

- Stage 1: node:20 builds the Vite bundle
- Stage 2: nginx-unprivileged on :8080, UID 101
- Proxies /api, SPA fallback, security headers

Compose stack

- postgres → backend → frontend
- each waits for the previous healthcheck
- one command: docker compose up --build



CI/CD pipeline

1 Test

Gitleaks · pytest · Alembic · ESLint · helm lint · tf validate

2 Build

Docker build, cached, both images

3 Scan

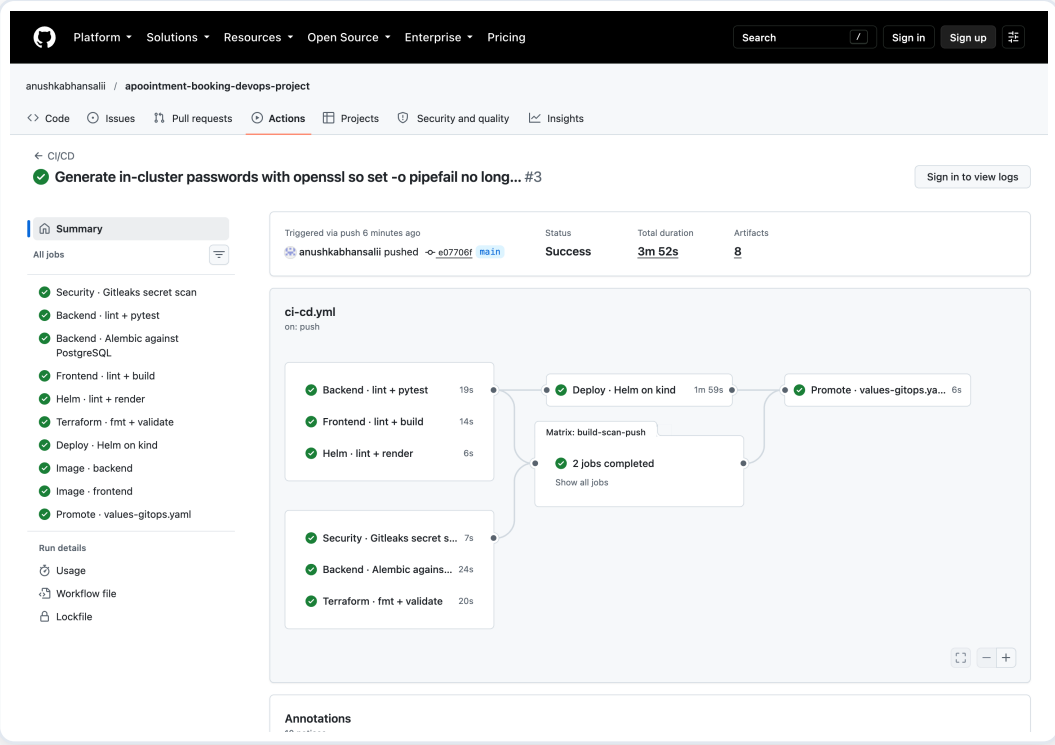
Trivy gate on HIGH/CRITICAL

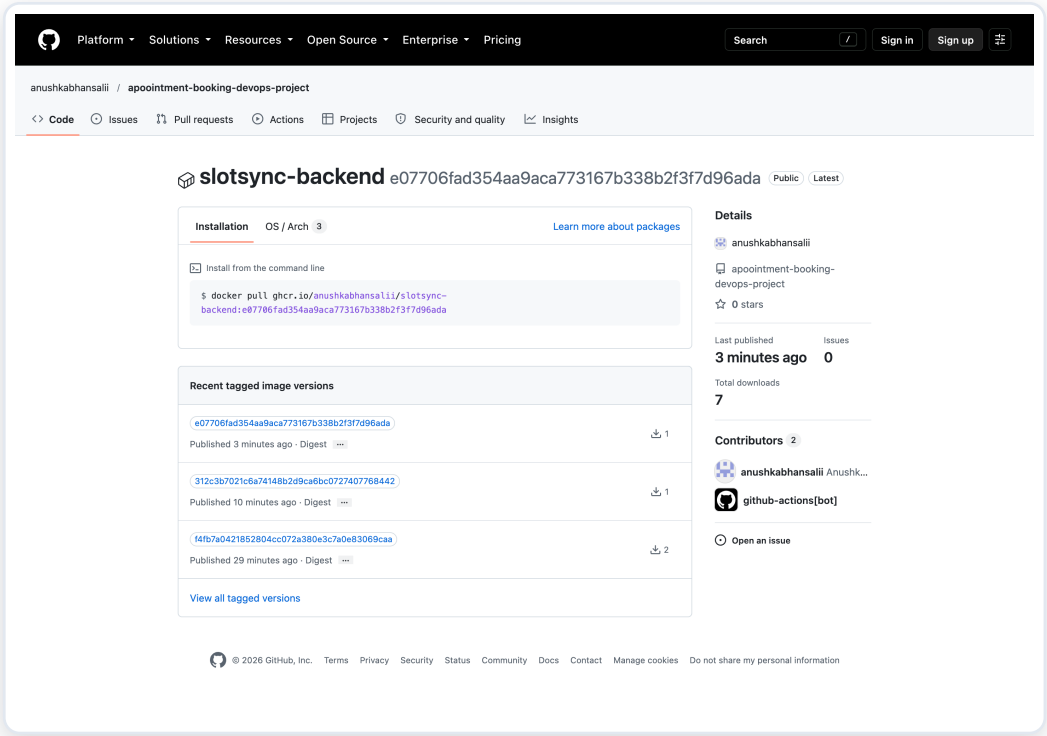
4 Push

GHCR :<git-sha>, amd64 + arm64

5 Deploy

kind smoke test, then GitOps promote





DevSecOps: the gate caught real CVEs

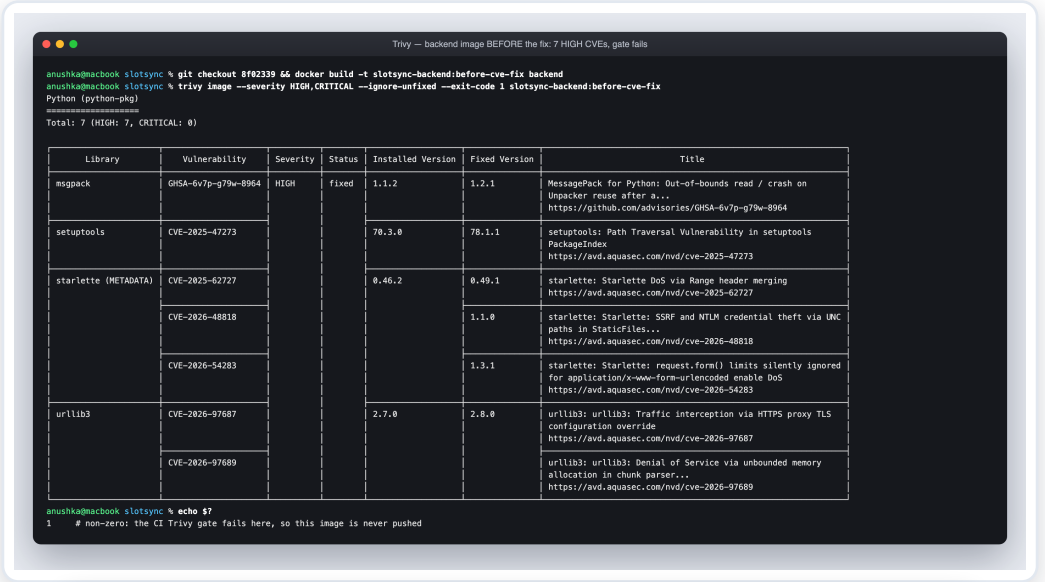
7 HIGH

first backend scan: Starlette, urllib3, msgpack, setuptools

0

after upgrading FastAPI/Starlette and removing pip from the image

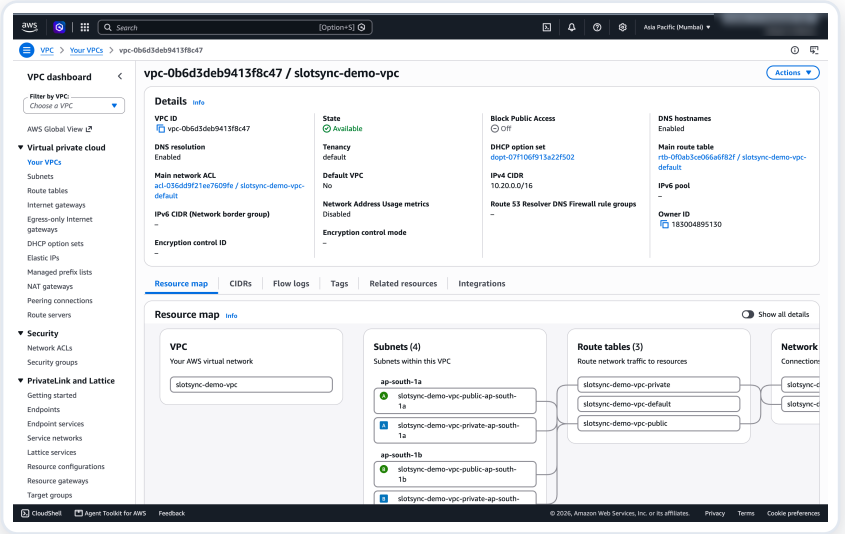
- **CVE-2025-62727:** a crafted Range header made Starlette merge ranges in quadratic time - one request could pin a CPU (DoS)
- **Gate:** Trivy fails the build on fixable HIGH/CRITICAL; SARIF goes to the GitHub Security tab
- **Gitleaks:** scans all Git history for credentials on every run
- **Runtime:** non-root, read-only root FS, capabilities dropped, Pod Security restricted, NetworkPolicy on Postgres

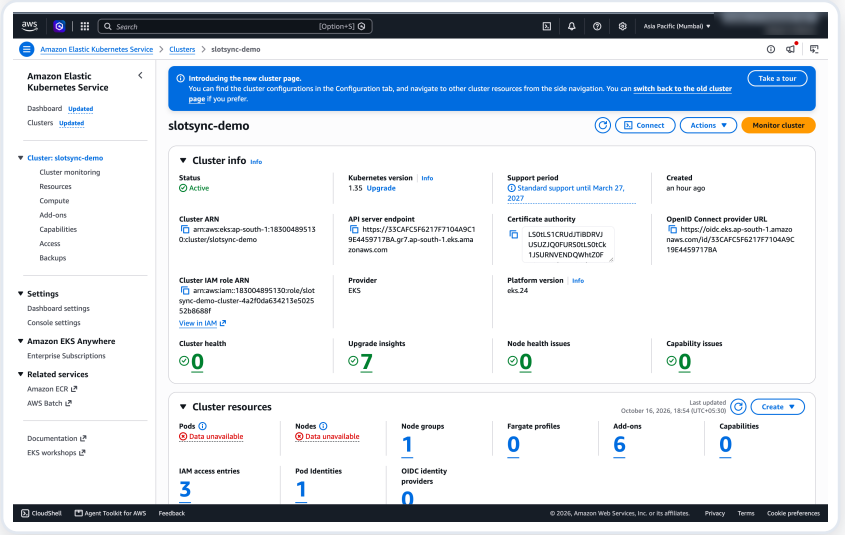




Infrastructure as code: AWS VPC + EKS

- **VPC 10.20.0.0/16:** 2 public subnets (NLB, NAT) and 2 private subnets (nodes) across 2 AZs
- **EKS 1.35:** managed node group of 2× m7i-flex.large, API restricted to my IP, secrets encrypted with KMS
- **Add-ons:** VPC CNI with NetworkPolicy, CoreDNS, kube-proxy, EBS CSI via Pod Identity, metrics-server
- **Workflow:** terraform plan → apply (61 resources) → destroy; state and tfvars git-ignored





61

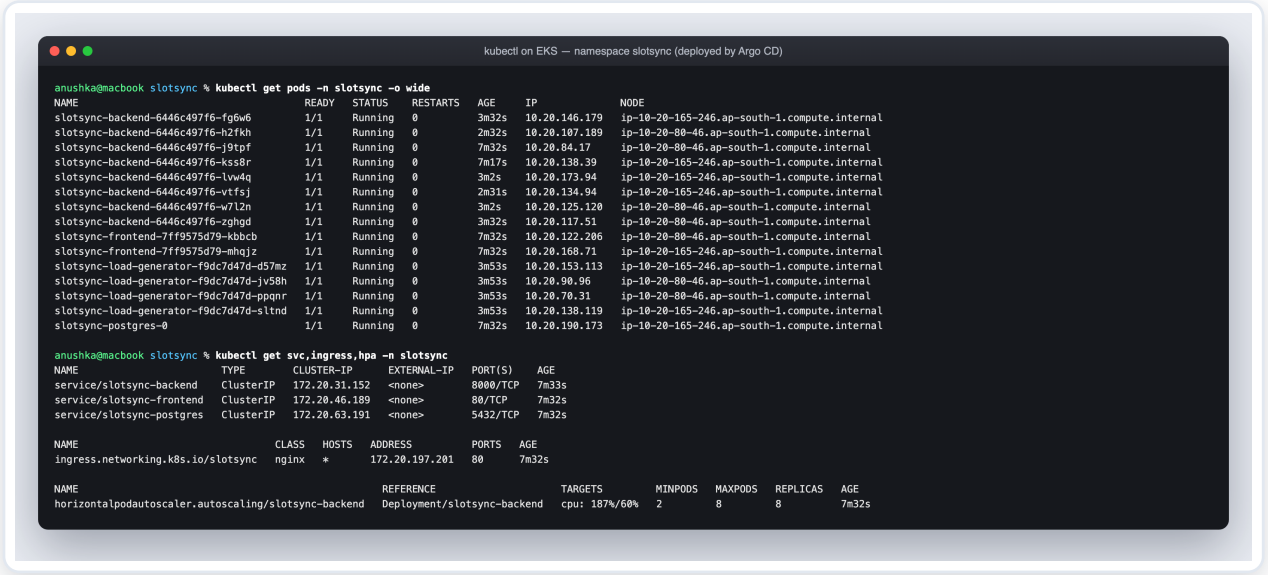
resources, 0 clicks

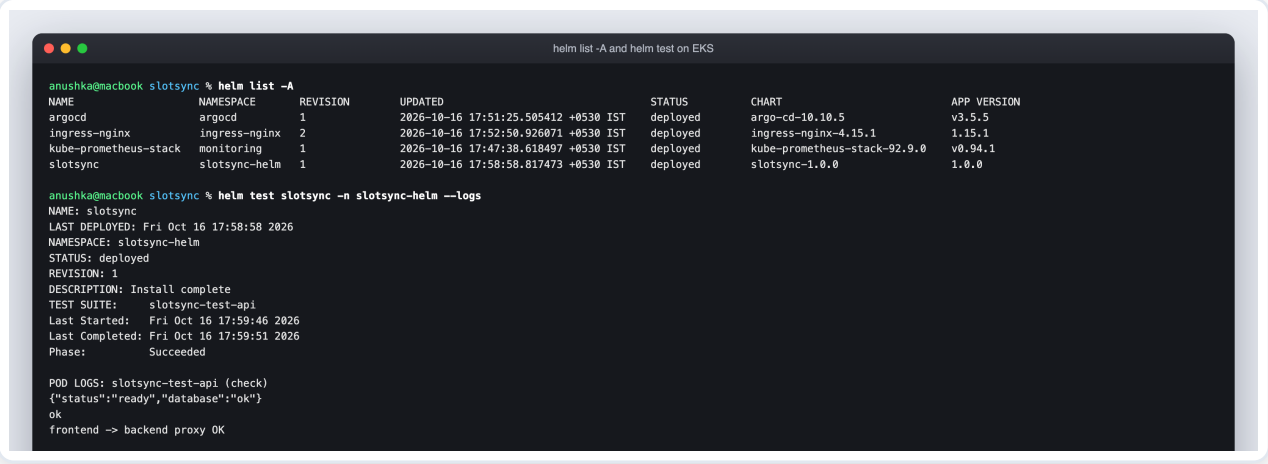
~$0.35/h

while running

Kubernetes and Helm

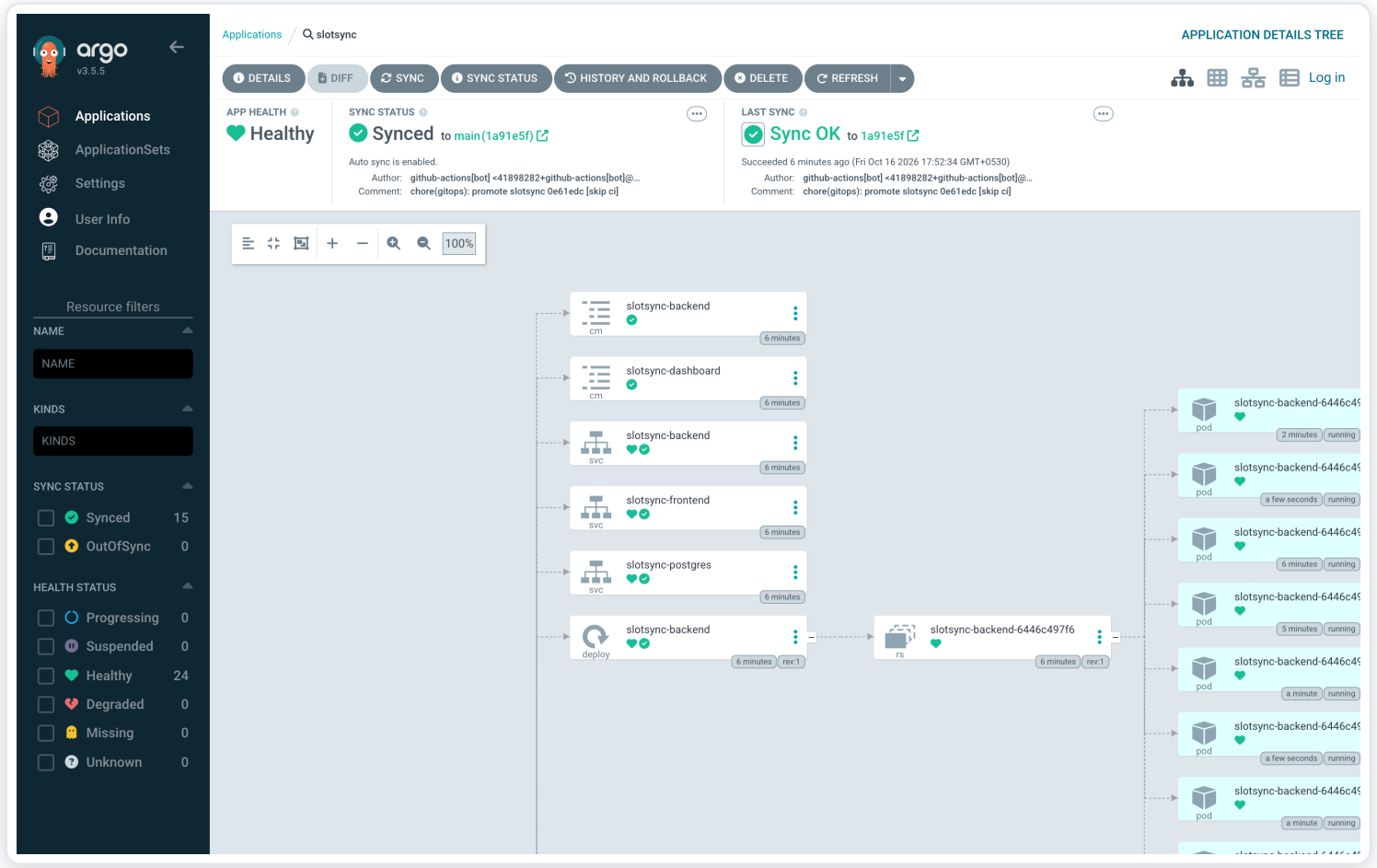
- **Chart helm/slotsync:** Deployments, Services, Postgres StatefulSet, Ingress, HPA, PDBs, NetworkPolicy, ServiceMonitor, alerts, dashboard
- **Ingress:** / → frontend, /api and /docs → backend via ingress-nginx (NLB-ready; this new AWS account can't create load balancers yet, so the demo reaches the controller by port-forward)
- **Resilience:** 2+ replicas, startup/liveness/readiness probes, zero-downtime rolling updates
- **HPA:** backend scales 2 → 8 on CPU; the EKS load test drove it to 8/8 pods
- **helm test:** checks /ready and the frontend → backend proxy path





GitOps with Argo CD

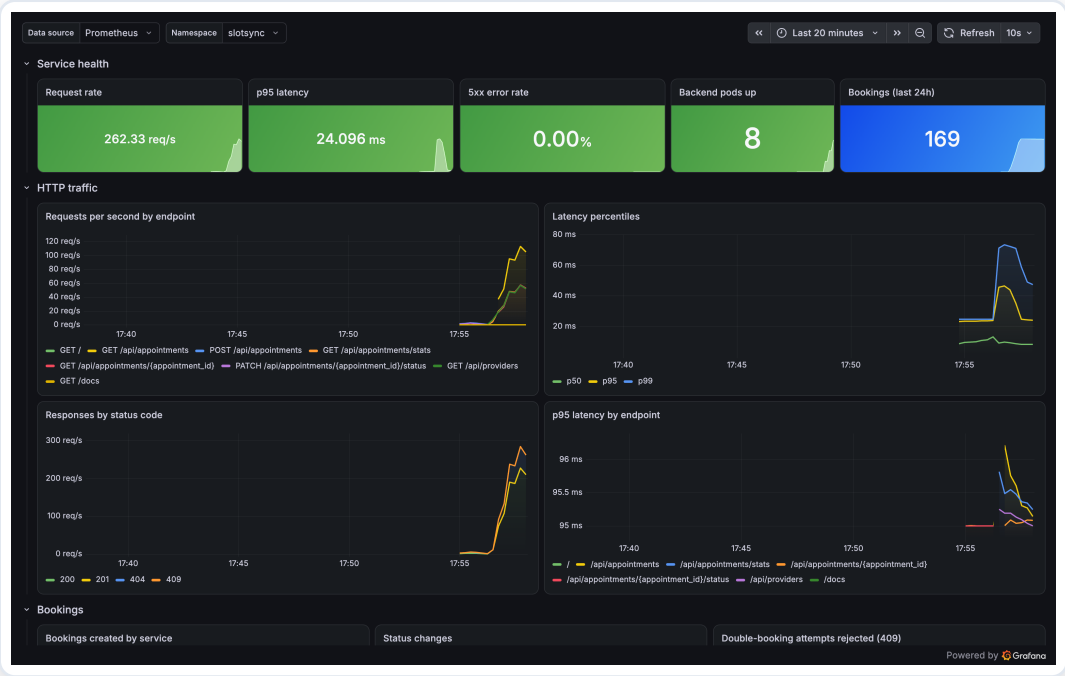
1. **git push**
2. **pipeline green**
3. **bot commits SHA**
4. **Argo CD syncs**
5. **rolling update**



Live on EKS: push eb90e7c → green pipeline → bot commit be5839a → Argo CD rolled all 8 backend pods with zero downtime. Rollback = git revert.

Observability

- **/metrics:** request count, latency histograms and business counters (bookings, status changes, rejected double-bookings)
- **Prometheus:** kube-prometheus-stack; a ServiceMonitor scrapes each backend pod every 15 s
- **Alerts:** backend down, 5xx above 5%, p95 latency above 500 ms
- **Grafana:** 19-panel dashboard generated from code and shipped by the Helm chart





Key learnings

Scanners earn their place

Trivy found 7 real HIGH CVEs on day one - two came from pip, which the app never needed at runtime.

Start-up order matters

Pods crash-looped while Postgres initialised; a wait-for-DB step and an advisory lock fixed restarts and migration races.

GitOps simplifies CD

CI only writes a SHA to Git. No kubeconfig in GitHub, and rollback is a git revert.

Cloud is more than code

An org SCP blocked EKS, a resolver dropped IAM calls, a new account could not create load balancers - read the error, then adapt.

Secure by default is cheap

Non-root, read-only FS and drop-all capabilities cost a few lines once; the namespace now rejects any pod that breaks them.

Measure what users feel

Latency and booking counters say more about the studio than CPU graphs alone.



Thank you

Next steps: Amazon RDS instead of in-cluster Postgres · TLS with ACM and Route 53 · OIDC login for staff · request-based autoscaling with KEDA

Repository github.com/anushkabhansalii/apoointment-booking-devops-project

Demo video demo/presentation.mp4

Evidence docs/EVIDENCE.md